
Module 10 — Search with Elasticsearch

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Why a Search Engine?	2
2.2 Analyzers	2
2.3 text or keyword?	3
2.4 Elasticsearch Is Not the Source of Truth	3
3. Step-by-Step Examples	3
3.1 Mapping a Document	4
3.2 The Repository	5
3.3 Full-Text Search with NativeQuery	5
3.4 Facets with Aggregations	6
3.5 The Search API	7
3.6 Keeping the Index in Sync with PostgreSQL	8
3.7 Testing with Testcontainers	9
4. Common Mistakes and Best Practices	10
5. Summary	10
6. Further Reading	11

1. Learning Goals

By the end of this module you will be able to:

- Explain how an inverted index and analyzers make full-text search fast
- Map a Java record to an index with `@Document` and `@Field`, including a Turkish analyzer
- Use `ElasticsearchRepository` for CRUD and derived queries
- Write bool queries with `NativeQuery`: full-text matching, field boosts, filters and highlighting
- Count results per category with aggregations (facets)
- Keep the search index in sync with PostgreSQL after each commit
- Test search code with `@DataElasticsearchTest` and `Testcontainers`

Prerequisites: Module 06 (PostgreSQL, transactions) · **Estimated time:** 5 hours · **Docker required** (give Docker at least 2 GB of memory for Elasticsearch)

2. Concepts

2.1 Why a Search Engine?

WHERE description LIKE '%kitapları%' in PostgreSQL reads every row, misses “Kitapları” (case) and “kitaplar” (another form of the same word), and cannot tell which result fits best. A search engine solves this with an **inverted index**: when a document is stored, its text is split into terms, and for every term the index remembers which documents contain it.

```
"Yazılım mimarisi üzerine kitaplar" → yazıl · mimaris · üzer · kitap
                                   |
inverted index: kitap → [doc 3]
                mimaris → [doc 3]
```

A search analyzes the query text the same way and looks the terms up. Each hit gets a **score** (BM25): rare terms and short fields count more.

2.2 Analyzers

An analyzer turns text into terms: split into words (tokenizer), lowercase, remove stop words, reduce words to their stem.

Text	standard analyzer	turkish analyzer
İstanbul'daki	istanbul'daki	istanbul

Text	standard analyzer	turkish analyzer
kitapçılar	kitapçılar	kitapç1
IŞIK	işik (wrong letter)	1ş1k

The turkish analyzer knows Turkish lowercasing (I → 1, not i), removes the suffix after an apostrophe and cuts off inflections. The stems do not have to be real words: they only have to be the same for the text and the query. Choose the analyzer per field, by the language of its content.

2.3 text or keyword?

text	keyword
Analyzed into terms	Stored as one exact value
Full-text search, relevance	Filters, sorting, aggregations
Title, description	Category, ISBN, author, status

A field can be both: a text field with a keyword sub-field (a multi-field).

2.4 Elasticsearch Is Not the Source of Truth

Elasticsearch has no transactions across documents, and a new document becomes searchable only after the next **refresh** (every second by default). Keep the data in PostgreSQL, and treat the index as a copy that can be rebuilt at any time.

3. Step-by-Step Examples

With Docker running, start the application. PostgreSQL and Elasticsearch start from the root `compose.yaml` (profiles `elastic` and `postgres`):

```
./mvnw -pl modules/10-elasticsearch/lesson spring-boot:run
```

On start, the tour (LessonTour) copies the six books from PostgreSQL into Elasticsearch and runs every example below.

i Note

This module's tables live in the PostgreSQL schema `search` (`spring.flyway.schemas` and `spring.datasource.hikari.schema`), so they never collide with the tables of modules 05 and 06 in the shared bookstore database.

3.1 Mapping a Document

```
@Document(indexName = "books")
public record BookDocument(
    @Id String id,
    @MultiField(mainField = @Field(type = FieldType.Text), //
        ↪ searchable words ...
        otherFields = @InnerField(suffix = "sort", type = FieldType.Keyword))
    ↪ // ... and sortable as a whole
    String title,
    @Field(type = FieldType.Keyword) String author, //
    ↪ exact values only
    @Field(type = FieldType.Text, analyzer = "turkish") String description,
    ↪ // Turkish stemming + lowercase
    @Field(type = FieldType.Keyword) String category, //
    ↪ filters and facets
    @Field(type = FieldType.Double) double price) {

    public static BookDocument from(Book book) {
        return new BookDocument(String.valueOf(book.id()), book.title(), book.author(),
            ↪ book.description(),
            book.category(), book.price().doubleValue());
    }
}
```

- `@Document(indexName = "books")`: Spring Data creates the index with this mapping when the repository starts, if it does not exist yet.
- `title` is a multi-field: `title` for full-text search, `title.sort` as a keyword for sorting.
- `description` uses the `turkish` analyzer, because the descriptions are Turkish.
- `author` and `category` are keyword: exact values for filters and facets.

Look at the mapping Spring Data created:

```
curl localhost:9200/books/_mapping
```

Spring Data also adds a `_class` field, which it uses to read documents back into the right Java type.

Test the analyzer directly:

```
curl -XPOST localhost:9200/_analyze -H 'Content-Type: application/json' \
  -d '{ "analyzer": "turkish", "text": "İstanbul\'\'daki kitapçılar" }'
# tokens: "istanbul", "kitapçı"
```

3.2 The Repository

```
public interface BookSearchRepository extends ElasticsearchRepository<BookDocument,
    ↪ String> {

    List<BookDocument> findByAuthor(String author);           // term query on the
    ↪ keyword field
}
```

ElasticsearchRepository works like the repositories of modules 05–07: save, findById, deleteAll, paging and derived queries. findByAuthor becomes a term query on the keyword field author.

3.3 Full-Text Search with NativeQuery

For real search pages, derived queries are not enough. NativeQuery gives the full query language of the Elasticsearch Java client:

```
public List<BookHit> search(String text, @Nullable String category, @Nullable Double
    ↪ maxPrice) {
    NativeQuery query = NativeQuery.builder()
        .withQuery(q -> q.bool(bool -> {
            bool.must(m -> m.multiMatch(match -> match
                .query(text)
                .fields("title^3", "description")));           // a title
            ↪ match counts 3x
            if (category != null) {
                bool.filter(f -> f.term(t -> t.field("category").value(category)));
            }
            ↪ // yes/no, no score
            if (maxPrice != null) {
                bool.filter(f -> f.range(r -> r.number(n ->
            ↪ n.field("price").lte(maxPrice))));
            }
            return bool;
        })))
        .withHighlightQuery(new HighlightQuery(new Highlight(List.of(
            new HighlightField("title"), new HighlightField("description")),
            ↪ BookDocument.class)))
```

```

        .build();
    return operations.search(query,
        ⇨ BookDocument.class).stream().map(BookHit::from).toList();
}

```

- A **bool** query combines clauses. **must** clauses must match and add to the score. **filter** clauses must match but do not change the score, and Elasticsearch can cache them.
- **multi_match** searches several fields. `title^3` makes a match in the title count three times as much.
- The **highlight** request returns the matching fragments with `<em>` tags around the matched words.

The tour output (the score is in brackets):

```

== 3.3 Full-text search
java: [Effective Java (3.36), Java Puzzlers (3.36), Spring in Action (0.65)]
kitap (text says "kitaplar"): [Clean Architecture (1.45)]
istanbul (text says "İstanbul"): [İstanbul Hatırası (5.03)]
java, programming, <= 60: [Java Puzzlers (3.36)]
highlight: [İstanbul sokaklarında geçen bir <em>polisiye</em> roman.]

```

“Spring in Action” has “Java” only in its description, so it scores far lower than the two books with “Java” in the title.

3.4 Facets with Aggregations

A search page usually shows how many results each category has. An **aggregation** computes this in the same request:

```

public Map<String, Long> categoryFacets(String text) {
    NativeQuery query = NativeQuery.builder()
        .withQuery(q -> q.multiMatch(match -> match.query(text).fields("title^3",
            ⇨ "description")))
        .withAggregation("categories", Aggregation.of(a -> a.terms(t ->
            ⇨ t.field("category"))))
        .withMaxResults(0) // only the counts,
            ⇨ no documents
        .build();
    SearchHits<BookDocument> hits = operations.search(query, BookDocument.class);

    var aggregations = (ElasticsearchAggregations)
        ⇨ Objects.requireNonNull(hits.getAggregations());
    List<StringTermsBucket> buckets =
        ⇨ Objects.requireNonNull(aggregations.get("categories"))
            .aggregation().getAggregate().sterms().buckets().array();
}

```

```

Map<String, Long> counts = new LinkedHashMap<>(); // biggest category
    ↪ first
buckets.forEach(bucket -> counts.put(bucket.key().stringValue(), bucket.docCount()));
return counts;
}

```

- A terms aggregation groups documents by the values of a keyword field, biggest group first.
- withMaxResults(0) returns only the counts, not the documents.

```
== 3.4 Facets
```

```
java per category: {programming=3}
```

3.5 The Search API

```

@RestController
@RequestMapping("/api/search")
class SearchController {

    private final BookSearch search;

    SearchController(BookSearch search) {
        this.search = search;
    }

    @GetMapping
    SearchResponse search(@RequestParam String q,
        @RequestParam(required = false) @Nullable String category,
        @RequestParam(required = false) @Nullable Double maxPrice) {
        return new SearchResponse(search.search(q, category, maxPrice),
            ↪ search.categoryFacets(q));
    }
}

```

```
curl 'localhost:8080/api/search?q=java&maxPrice=90'
```

```

{"hits":[{"id":"1","title":"Effective Java", ... ,"highlights":["<em>Java</em> dilinde
    ↪ etkili programlama için doksan kural.", "Effective <em>Java</em>"]},
    {"id":"5","title":"Java Puzzlers", ...}],
"categories":{"programming":3}}

```

The hits respect maxPrice (2 books), but the facet counts all 3 matches for “java”, because categoryFacets does not know the filters. Exercise 2 builds a facet search where this is done on purpose, and correctly.

3.6 Keeping the Index in Sync with PostgreSQL

The catalog is written to PostgreSQL. Every write publishes a `BookSaved` event inside the transaction:

```
@Service
public class BookStore {

    private final JdbcClient jdbc;
    private final ApplicationEventPublisher events;

    public BookStore(JdbcClient jdbc, ApplicationEventPublisher events) {
        this.jdbc = jdbc;
        this.events = events;
    }

    @Transactional
    public Book add(NewBook book) {
        Book saved = insert(book);
        events.publishEvent(new BookSaved(saved));           // delivered after the
    ↪ commit (BookIndexer)
        return saved;
    }
}
```

A `@TransactionalEventListener` receives the event **only after the commit**:

```
@Component
public class BookIndexer {

    private final BookSearchRepository index;
    private final BookStore store;

    public BookIndexer(BookSearchRepository index, BookStore store) {
        this.index = index;
        this.store = store;
    }

    @TransactionalEventListener                               // default phase:
    ↪ AFTER_COMMIT
    public void onBookSaved(BookSaved event) {
        index.save(BookDocument.from(event.book()));         // rolled-back
    ↪ books never get here
    }

    public long reindexAll() {                                // rebuild the
    ↪ index from the source of truth
        index.deleteAll();
    }
}
```

```

        index.saveAll(store.findAll().stream().map(BookDocument::from).toList());
        return index.count();
    }
}

```

- A transaction that rolls back never reaches the index. `BookIndexSyncIT` checks this: `addAll` inserts one new book, then fails on a duplicate ISBN, and the new book never becomes searchable.
- `reindexAll` rebuilds the index from PostgreSQL. The tour runs it on start.

! Warning

After the commit, nothing can roll back anymore. If Elasticsearch is down at that moment, the event is lost and the index misses the book. Reindexing regularly repairs this. For a guarantee, write the event to an **outbox** table in the same transaction and deliver it with a message broker (module 11, Kafka).

3.7 Testing with Testcontainers

```

@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    static final ElasticsearchContainer ELASTICSEARCH = new ElasticsearchContainer(
        DockerImageName.parse("elasticsearch:9.4.5")

        ↪ .asCompatibleSubstituteFor("docker.elastic.co/elasticsearch/elasticsearch"))
        .withEnv("xpack.security.enabled", "false") // plain HTTP, no
        ↪ password: tests only
        .withEnv("ES_JAVA_OPTS", "-Xms512m -Xmx512m");

    static final PostgreSQLContainer POSTGRES = new PostgreSQLContainer(
        DockerImageName.parse("pgvector/pgvector:0.8.6-
    ↪ pg18").asCompatibleSubstituteFor("postgres"));

    @Bean
    @ServiceConnection
    ElasticsearchContainer elasticsearchContainer() {
        return ELASTICSEARCH;
    }

    @Bean
    @ServiceConnection
    PostgreSQLContainer postgresContainer() {
        return POSTGRES;
    }
}

```

}

- The compose image is called `elasticsearch`, the `Testcontainers` class expects `docker.elastic.co/elasticsearch/elasticsearch:asCompatibleSubstituteFor` connects the two.
- `xpack.security.enabled=false` gives plain HTTP without a password, as in `compose.yaml`. Never do this outside local development.
- `@DataElasticsearchTest` loads only the Elasticsearch infrastructure and repositories. `BookSearchTest` saves its documents in `@BeforeEach` and calls `indexOps(...).refresh()`, so that they are searchable at once.

★ **Tip**

Without the refresh, a test may search before Elasticsearch has made the new documents visible, and fail only sometimes.

4. Common Mistakes and Best Practices

!! Caution

Do not use Elasticsearch as your only database. Keep the source of truth in a transactional database, and make the index rebuildable from it.

- **Do:** choose keyword for values you filter, sort or aggregate on. Aggregating on a text field fails.
- **Don't:** put conditions that should not change the ranking into must. Use filter: it is faster and cacheable.
- **Do:** pick the analyzer by the language of the field. Turkish text with the standard analyzer misses inflected forms and turns I into i instead of 1.
- **Don't:** change the mapping of an existing field. Create a new index with the new mapping and reindex (exercise 3).
- **Do:** refresh the index in tests after writing, and never call refresh after every write in production.
- **Don't:** disable security (`xpack.security.enabled=false`) anywhere but on your own machine.

5. Summary

- An inverted index and analyzers make full-text search fast and language-aware. Every hit has a relevance score.
- `@Document` and `@Field` define the mapping: text for searching, keyword for filters and facets, analyzers per field.

- ElasticsearchRepository covers CRUD and simple queries. NativeQuery gives bool queries, boosts, filters and highlighting.
- Aggregations compute facets in the same request.
- PostgreSQL stays the source of truth. @TransactionalEventListener indexes after the commit, and a reindex repairs any gap.
- @DataElasticsearchTest with a Testcontainers Elasticsearch tests search code against the real engine.

Next step: [Exercises](#)

6. Further Reading

- [Spring Data Elasticsearch Reference · Object Mapping](#)
- [Spring Boot — Elasticsearch](#)
- [Elasticsearch — Language Analyzers](#)
- [Elasticsearch — Bool Query · Highlighting](#)
- [Elasticsearch — Terms Aggregation](#)
- [Spring Framework — Transaction-bound Events](#)